Tecnicatura Universitaria en Programación a Distancia
Universidad Tecnológica Nacional

Programación 1 — 1° Cuatrimestre 2026

# Trabajo Práctico Integrador

## Sistema de Control de Inventario — Ferretería

*Informe de diseño y fundamentación*


| | |
|---|---|
| **Alumno** | Federico Barranco |
| **Comisión** | A26 C1-02 |
| **Tutora** | Sofía Elizabeth Fernández |
| **Colaboración inicial** | Nicolás Tissoni (primera versión de las opciones 1, 2, 3 y 5) |
| **Fecha de entrega** | 03/11/2026 |
| **Repositorio** | github.com/fedebarranco/TP_Final-Sistema_de_Control_de_Inventario |

## 1. Qué hace el programa y cómo se ejecuta

El programa lleva el control de stock de una ferretería. Muestra un menú de siete opciones que se repite después de cada operación, hasta que el usuario elige salir. Los datos se guardan en memoria, en la lista `inventario`, mientras el programa está abierto: cada venta o ingreso cambia el stock en el momento, y al cerrar el programa no queda nada guardado.

El código está dividido en tres archivos:

| Archivo | Qué tiene |
|---|---|
| `main.py` | `main()`: crea el inventario y repite el menú. **Es el archivo que se ejecuta para iniciar el programa** (`python main.py`). |
| `carga_y_consulta.py` | Opciones 1, 2, 3 y 5, y las funciones auxiliares `validar_nombre` y `buscar_herramienta`. |
| `stock_y_menu.py` | Opciones 4 y 6, y `mostrar_menu`, que imprime el texto del menú. |

`main.py` trae las funciones de los otros dos archivos con `from ... import ...`.

## 2. Conceptos aplicados

### 2.1 Listas

Una lista es una colección ordenada de elementos que se puede modificar: se le pueden agregar elementos, cambiar los que tiene y recorrerla. En el programa hay una sola lista, `inventario`, que se crea vacía dentro de `main()`:

```
inventario = []
```

Las opciones 1 y 5 le agregan herramientas al final con `append()`, y las opciones 2, 3, 4 y 6 la recorren con `for herramienta in inventario`. Para saber si todavía no se cargó nada uso `len(inventario) == 0`. Como la consigna no permite otras colecciones, el reporte de agotados no arma una lista aparte con los resultados: recorre `inventario` y va imprimiendo los que tienen stock 0.

Un detalle importante es que las listas son mutables: cuando le paso `inventario` a una función, la función recibe la misma lista, no una copia. Por eso el `append()` que se hace dentro de `cargar_herramientas` se ve después en `main()`, sin tener que devolver nada con `return`.

### 2.2 Diccionarios

Un diccionario guarda pares *clave: valor*, y a cada valor se accede por su clave. En el programa cada herramienta es un diccionario con exactamente dos claves: `'herramienta'` (el nombre) y `'cantidad'` (el stock). Por ejemplo, después de cargar dos herramientas la lista queda así:

```
[{'herramienta': 'Martillo', 'cantidad': 10}, {'herramienta': 'Pala', 'cantidad': 0}]
```

El diccionario de una herramienta nueva se arma directamente adentro del `append()`, sin guardarlo antes en una variable suelta:

```
inventario.append({'herramienta': nombre_herramienta_ingresada, 'cantidad':
stock_herramienta_ingresada})
```

Para cambiar el stock en la opción 6 modifico el valor de la clave `'cantidad'`. La función `buscar_herramienta` devuelve el mismo diccionario que está dentro de la lista, así que el cambio queda guardado en `inventario`:

```
herramienta['cantidad'] = stock_resultante
```

### 2.3 Funciones

Una función es un bloque de código con nombre que se define con `def`, recibe datos por parámetro y puede devolver un resultado con `return`. Dividí el programa para que cada opción del menú tenga su propia función:

| Opción | Función | Archivo |
|---|---|---|
| 1. Carga de herramientas | `cargar_herramientas(inventario)` | carga_y_consulta.py |
| 2. Ver inventario | `mostrar_inventario(inventario)` | carga_y_consulta.py |
| 3. Consultar stock | `consultar_stock(inventario)` | carga_y_consulta.py |
| 4. Reporte de agotados | `reporte_agotados(inventario)` | stock_y_menu.py |
| 5. Alta de producto | `alta_producto(inventario)` | carga_y_consulta.py |
| 6. Actualizar stock | `actualizar_stock(inventario)` | stock_y_menu.py |
| 7. Salir | se resuelve en `main()`: termina el `while` | main.py |

Además hay dos funciones auxiliares que se usan en las opciones 1, 3, 5 y 6. `validar_nombre(nombre)` le saca los espacios del principio y del final con `strip()` y, si el nombre queda vacío, lanza un error. `buscar_herramienta(inventario, nombre)` recorre la lista comparando los nombres con `lower()` (así "Martillo" y "martillo" son la misma herramienta) y devuelve el diccionario encontrado, o `None` si no está. Así la regla de cómo se comparan los nombres está escrita en un solo lugar y no repetida cuatro veces.

Todas las funciones que usan el inventario lo reciben como parámetro: no hay variables globales. Cada función tiene un docstring que explica qué hace.

### 2.4 Condicionales

Los condicionales (`if`, `elif`, `else`) ejecutan una parte del código u otra según si se cumple una condición. En `main()` una cadena de `if/elif` elige qué función llamar según la opción, y la última rama, `elif opcion != 7`, lanza el error de opción fuera de rango. Dentro de las funciones los condicionales comprueban las reglas del negocio:

- `cantidad_de_herramientas_ingresadas <= 0` y `stock < 0` en la carga y el alta.
- `herramienta is None` para saber si la herramienta no existe (opciones 3 y 6).
- `tipo != "1" and tipo != "2"` para rechazar un movimiento que no es venta ni ingreso.
- `stock_resultante < 0` para no permitir una venta mayor al stock.
- En el reporte de agotados, `herramienta['cantidad'] == 0` decide qué se muestra, y un contador sirve para elegir entre la lista de agotados y el mensaje "Ninguna herramienta tiene stock cero".

### 2.5 Bucles

Usé `while` cuando la repetición depende de una condición y `for` cuando se sabe de antemano cuántas veces hay que repetir:

- `while opcion != 7` en `main()`: hace que el menú sea persistente.
- `while True` con `break` en la opción 1: vuelve a pedir un dato hasta que sea válido. El `break` se ejecuta solo si no hubo error.
- `for _ in range(cantidad_de_herramientas_ingresadas)`: carga exactamente la cantidad de herramientas que indicó el usuario.
- `for herramienta in inventario`: recorre la lista para mostrar, buscar o contar.

En la opción 1 es clave dónde está el `append()`: tiene que estar adentro del `for`, para que se agregue una herramienta por vuelta. En la primera versión del código había quedado afuera del `for` y solo se guardaba la última herramienta cargada; además, si la cantidad era 0, el programa se cortaba. Lo encontré probando el programa y lo corregí.

### 2.6 Manejo de errores con try/except y raise

Hay dos situaciones distintas. Si el usuario escribe algo que no es un número donde va un número (por ejemplo "abc" o "3.5"), `int()` ya lanza un `ValueError` y lo atrapo con `try/except`. Si el dato es válido pero no cumple una regla del negocio (nombre vacío o repetido, stock negativo, unidades que no son mayores que 0, una venta mayor al stock, opción fuera de rango), lo compruebo con un `if` y lanzo el error yo con `raise ValueError("mensaje")`. Por ejemplo, en `actualizar_stock`:

```
stock_resultante = herramienta['cantidad'] - unidades
if stock_resultante < 0:
    raise ValueError(f"No hay stock suficiente: quedan {herramienta['cantidad']}
unidades")
```

Lo importante es dónde se atrapa cada error:

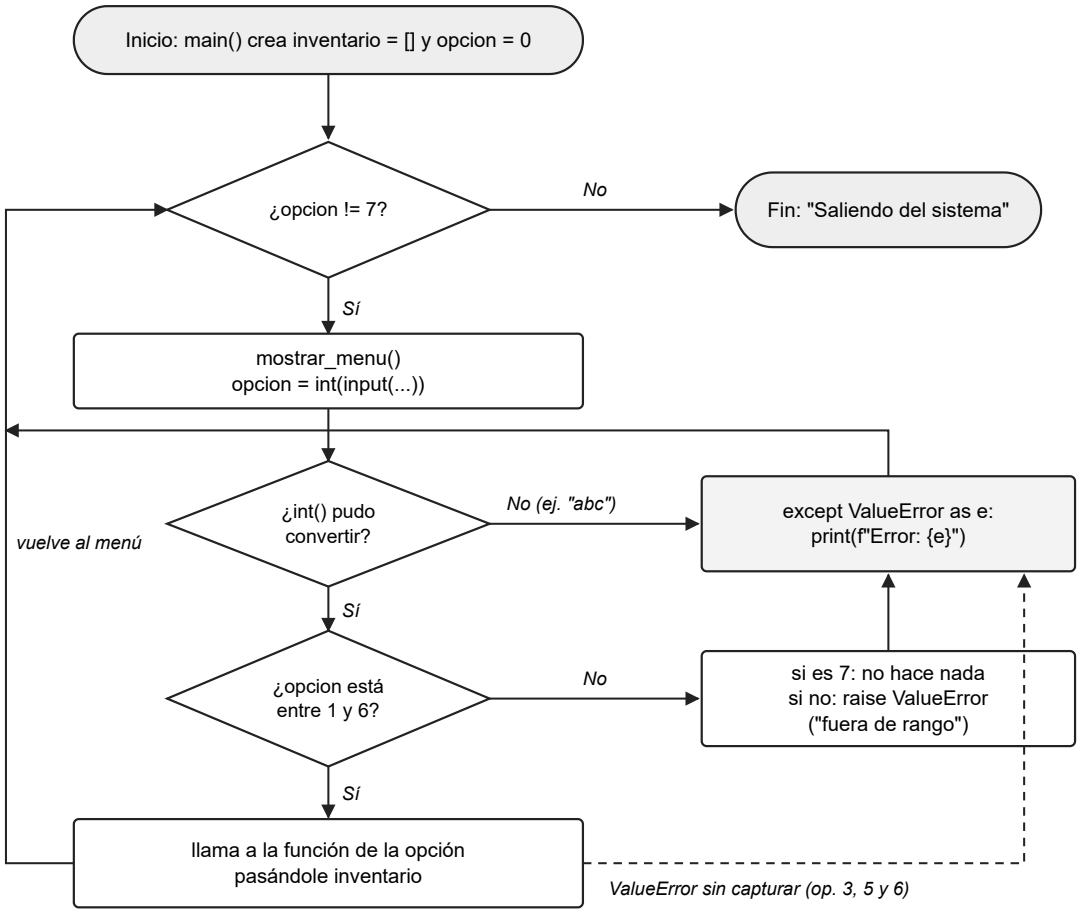
| Dónde | Cómo se maneja |
|---|---|
| Opción 1 (carga) | El error se atrapa adentro de la función, dentro de un `while True`, y se vuelve a pedir solo el dato que falló. Si el stock es inválido, el nombre que ya era válido se conserva. |
| Opciones 3, 5 y 6 | La función no atrapa el error: sale de la función y lo recibe el `except ValueError as e` de `main()`, que muestra `print(f"Error: {e}")` y vuelve al menú. En el alta y en actualizar stock, si `int()` falla, atrapo el error y lo vuelvo a lanzar con `raise` para que el mensaje esté en español. |
| Menú | Si la opción no es un número, el `except` de `main()` atrapa el error de `int()`. Si es un número fuera de 1 a 7, se lanza con `raise`. |
| Avisos | Inventario vacío, herramienta que no existe y tipo de movimiento inválido no son errores del dato sino avisos: se resuelven con un `if` y un `print`, sin excepción. |

Además, en todas las opciones el `append()` o el cambio de stock va al final, después de todas las validaciones. Así un dato inválido nunca deja una herramienta a medio cargar ni cambia un stock.

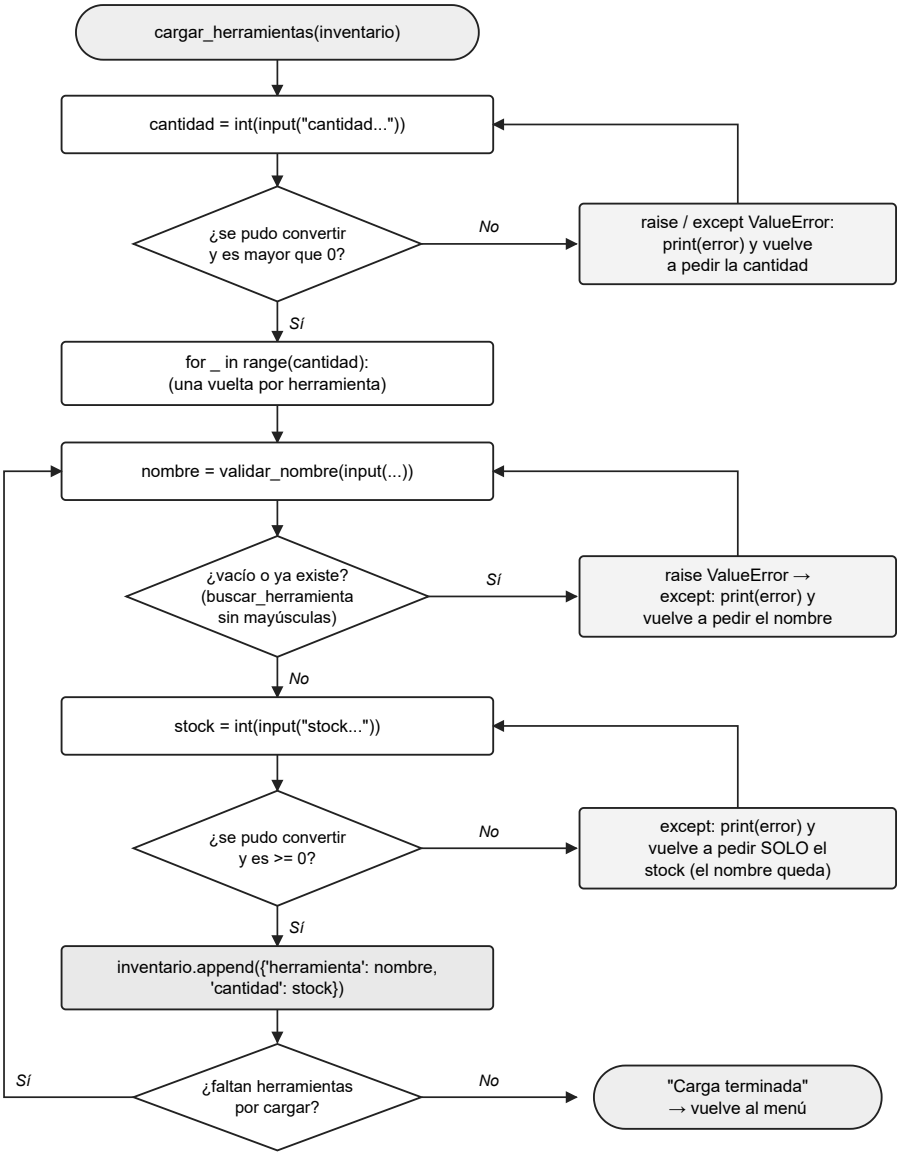
## 3. Diagramas de flujo

Los rombos son decisiones y las cajas grises son los lugares donde se maneja un error. La flecha punteada muestra el camino de un error que sale de una función sin ser atrapado.

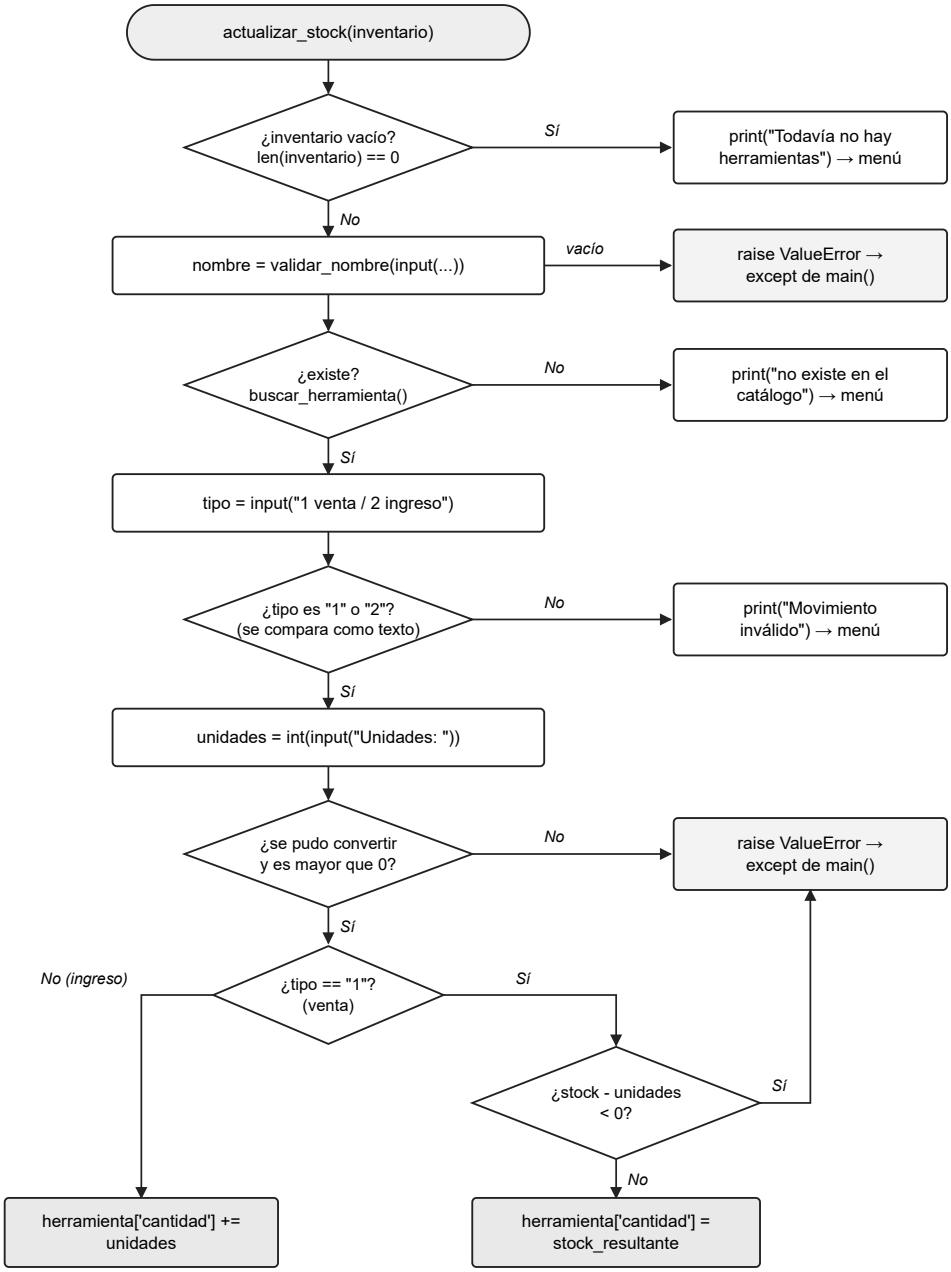
### 3.1 Menú principal — main()



## 3.2 Carga de herramientas y validación de datos — opción 1



## 3.3 Actualización de stock — opción 6




En los dos casos se muestra el stock nuevo y se vuelve al menú.

## 4. Lenguaje, entorno y recursos utilizados

**Python 3.14.7.** Es un lenguaje de programación interpretado y de alto nivel, con una sintaxis simple que usa la sangría para marcar los bloques. Es el lenguaje que se usa en la materia. Instalé la última versión estable porque prefiero trabajar siempre con la versión más actual. El programa no usa librerías externas: solo funciones que ya vienen con Python (`input`, `print`, `int`, `len`, `range`) y los métodos de texto `strip()` y `lower()`. Para repasar cómo funcionan esos métodos y las excepciones consulté la documentación oficial (docs.python.org, tutorial "Errores y excepciones").

**Visual Studio Code 1.142.0.** Es un editor de código gratuito de Microsoft. Con la extensión de Python se puede escribir, ejecutar y depurar el programa en el mismo lugar, y tiene una terminal integrada. Lo elegí porque es el editor que usa la cátedra (el video del TPI se graba ahí) y porque es el que uso todos los días en mi trabajo.

**Git 2.55.0 y GitHub Desktop 3.6.6.** Git es un sistema de control de versiones: guarda cada cambio del código en un *commit* con un mensaje, y permite volver atrás o ver quién hizo qué. GitHub Desktop es una aplicación con interfaz gráfica para usar Git sin escribir comandos. Los vimos en la Unidad 4 de la materia, y los usé para registrar cada corrección en un commit separado.

**GitHub.** Es una plataforma para guardar repositorios de Git en la nube. El repositorio del trabajo es público y tiene un README con las instrucciones para ejecutarlo. Al principio fue la forma de trabajar en equipo con Nicolás, y en su historial quedó registrado qué parte hizo cada uno.

## 5. Nota sobre la autoría

El trabajo empezó en pareja con Nicolás Tissoni, que escribió la primera versión de `cargar_herramientas`, `mostrar_inventario`, `consultar_stock` y `alta_producto`. En octubre él dejó la materia y terminé el trabajo solo, como permite la cátedra: corregí esas funciones para que cumplan la consigna vigente (comparar nombres sin mayúsculas ni espacios, el orden de los pasos del alta y el error del `append` en la carga) y escribí las funciones auxiliares, `reporte_agotados`, `actualizar_stock`, el menú y `main()`. La entrega es individual.